

FIAP / NAVEGANDO PELO MUNDO DEVOPS

Cidades ESG Inteligentes

EnergiaApi: CI/CD, containerização e orquestração

INTEGRANTES

Henrique Cole Fernandes, RM559957

GRUPO

Grupo 15

APLICAÇÃO BASE

ASP.NET Core 8 (C#), webservice de eficiência energética

REPOSITÓRIO

github.com/HenriqueCole/EnergiaApi

1. O projeto

A EnergiaApi é um webservice de eficiência energética e sustentabilidade escrito em ASP.NET Core 8, reaproveitado dos desafios anteriores da trilha. Ela monitora o consumo de energia de equipamentos urbanos, dispara alerta quando o consumo passa do limite configurado e monta relatórios agregados. É a base de dados que uma cidade usaria para decidir onde cortar desperdício.

Nesta fase, o objetivo não era mexer nas regras de negócio e sim colocar a aplicação para funcionar como funcionaria de verdade: build, teste, empacotamento e entrega em dois ambientes, tudo automatizado, com a configuração saindo do código para poder variar por ambiente.

Endpoints

Método	Rota	Descrição	JWT
POST	/api/auth/login	Autentica e devolve o token JWT	Não
GET	/api/consumos	Lista paginada de consumos	Não
POST	/api/leituras	Registra leitura e dispara alerta automático	Sim
GET	/api/relatorios/equipamento/{id}	Relatório agregado por equipamento	Não
GET	/api/alertas	Lista paginada de alertas	Sim
GET	/health	Status, ambiente e versão da instância. Criado para o pipeline	Não

O que entrou nesta fase

Arquivo	Para que serve
.github/workflows/ci-cd.yml	Pipeline com 4 jobs: build e testes, imagem, deploy em staging, deploy em produção
EnergiaApi.Api/Dockerfile	Imagem em dois estágios, com usuário sem privilégios e configuração por variável de ambiente
docker-compose.yml	Orquestração da API com o SQL Server, parametrizada por variáveis
.env.example, .env.staging, .env.production	Configuração de cada ambiente, fora do código
scripts/smoke-test.sh	Testa cada deploy depois que ele sobe
/health na API	Ponto de verificação do pipeline e das evidências

Estrutura do repositório

EnergiaApi/	
├ .github/workflows/ci-cd.yml	Pipeline de build, teste e deploy
└ EnergiaApi.Api/	API (Models, ViewModels, Data, Services, Controllers, Migrations)

		└ Dockerfile	Imagem da aplicacao, em dois estagios
		└ EnergiaApi.Tests/	6 testes de integracao com xUnit
		└ frontend/	Aplicacao React (Vite e TypeScript)
		└ postman/	Colecao e environment do Postman
		└ scripts/smoke-test.sh	Validacao de cada deploy
		└ docker-compose.yml	Orquestracao (volume, rede, variaveis)
		└ .env.example .env.staging .env.production	
		└ README.md	

2. O pipeline

A ferramenta escolhida foi o GitHub Actions. O motivo é prático: ele fica no mesmo lugar do código, não tem servidor de build para manter, os runners já vêm com Docker e Compose instalados, e o recurso de Environments resolve separação de segredos e aprovação manual sem precisar de mais nada.

As imagens vão para o GitHub Container Registry, que também é integrado ao repositório e autentica com o `GITHUB_TOKEN` do próprio run.

O pipeline inteiro roda no `push` da `main`. Em `pull_request` para a `main`, só o build e os testes rodam, para barrar código quebrado antes do merge. O `workflow_dispatch` permite disparar na mão.

Etapas

```
push na main
|
|→ 1. build-test ..... restore, build Release, dotnet test (xUnit), artefato .trx
|
|→ 2. image ..... build da imagem, push no GHCR (tags :sha-curto e :latest)
|
|→ 3. deploy-staging ..... environment "staging", pull, compose up, smoke test
|
|→ 4. deploy-production .... environment "production" com aprovacao manual, mesma imagem,
smoke test
```

#	Job	O que faz	Por quê
1	build-test	Roda <code>dotnet restore</code> , <code>dotnet build -c Release</code> e <code>dotnet test</code> com os 6 testes de integração. O relatório <code>.trx</code> fica salvo como artefato do run.	Se a compilação ou um teste falhar, nenhum job depois dele começa.
2	image	Constrói a imagem pelo Dockerfile e publica no GHCR com as tags <code>:<sha-curto></code> e <code>:latest</code> , aproveitando o cache de camadas do Actions (<code>type=gha</code>).	Gera um artefato único, identificado pelo commit, que os dois ambientes vão usar.
3	deploy-staging	Login no GHCR, <code>docker compose --env-file .env.staging -p energia-staging up -d --no-build</code> e smoke test na porta 8081. No fim imprime os logs da aplicação.	Testa a imagem em um ambiente igual ao de produção antes de qualquer coisa chegar no usuário.
4	deploy-production	Só roda se staging passou (<code>needs: deploy-staging</code>) e sobe a mesma imagem com <code>.env.production</code> na porta 8082, seguido do smoke test.	Produção recebe exatamente o binário que passou em staging, sem rebuild. Aquele bug de "mas funcionava em staging" simplesmente não existe.

Testes

A suíte xUnit sobe a API em memória com `WebApplicationFactory` e banco `InMemory`. São 6 testes, cobrindo os 5 endpoints de negócio e o `/health`:

```
$ dotnet test
Passed! - Failed: 0, Passed: 6, Skipped: 0, Total: 6, Duration: 586 ms
```

Fora isso, cada deploy passa pelo smoke test, que testa o ambiente já publicado: espera o `/health` responder, autentica em `/api/auth/login`, consulta `/api/consumos` (é aqui que se prova que a API fala com o banco e que as migrations rodaram) e chama a rota protegida `/api/alertas` com o token. Qualquer passo que falhe derruba o job.

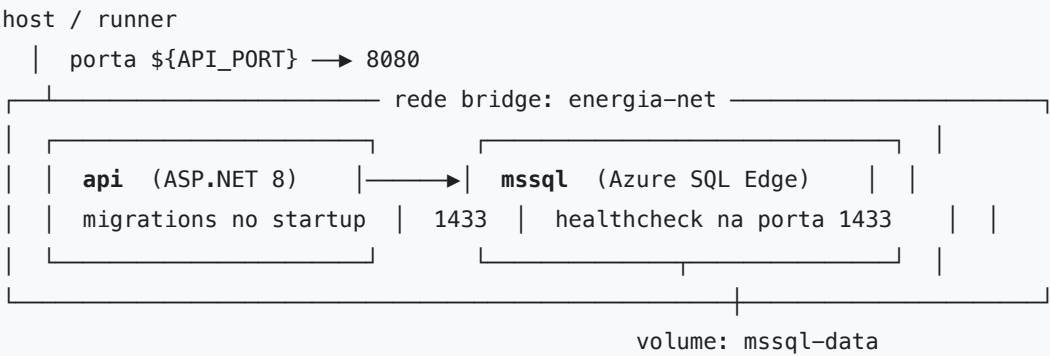
Ambientes e segredos

Os dois deploys usam GitHub Environments, um `staging` e outro `production`. Com isso, a URL de cada ambiente aparece na tela do run, os segredos ficam separados por ambiente, e a produção pode exigir aprovação manual. O `DB_PASSWORD` e o `JWT_KEY` entram no container como variável de ambiente, então nunca aparecem no código nem dentro da imagem.

A aprovação manual é configurada em Settings, Environments, production, Required reviewers. Se os segredos não estiverem cadastrados, o workflow usa os valores dos arquivos `.env.*` e roda do mesmo jeito. Fiz assim para o projeto poder ser clonado e testado sem configuração prévia.

3. Docker

Como o ambiente está montado



Dockerfile

```
# Estagio 1: build e publish. Usa o SDK, que e uma imagem grande, e some no final.
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

# Copia so o .csproj primeiro. Enquanto as dependencias nao mudam,
# o Docker reaproveita a camada de restore que ja esta em cache.
COPY ["EnergiaApi.Api/EnergiaApi.Api.csproj", "EnergiaApi.Api/"]
RUN dotnet restore "EnergiaApi.Api/EnergiaApi.Api.csproj"

COPY . .
RUN dotnet publish "EnergiaApi.Api/EnergiaApi.Api.csproj" \
    -c Release --no-restore -o /app/publish

# Estagio 2: runtime. So o ASP.NET Core, sem SDK e sem codigo-fonte.
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
WORKDIR /app
COPY --from=build /app/publish .

ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080

# Roda com usuario sem privilegios. O APP_UID=1654 ja vem definido na imagem base.
USER $APP_UID

ENTRYPOINT ["dotnet", "EnergiaApi.Api.dll"]
```

Estratégia	Como	Resultado
Build em dois estágios	Estágio <code>build</code> com o SDK, estágio <code>final</code> só com o runtime	A imagem entregue não leva SDK, compilador nem código-fonte. Fica mais leve e com menos coisa exposta
Cache de camadas	<code>COPY</code> do <code>.csproj</code> e <code>restore</code> antes do código	Mexer no código não re-baixa os pacotes NuGet. No pipeline o cache também é reaproveitado, com <code>type=gha</code>

Usuário sem privilégios	<code>USER \$APP_UID</code>	O processo não roda como root dentro do container
<code>.dockerignore</code>	Ignora <code>bin</code> , <code>obj</code> , <code>node_modules</code> e <code>frontend</code>	Contexto de build menor e build que dá o mesmo resultado sempre
Configuração por fora	Nenhuma senha ou connection string dentro da imagem	A mesma imagem roda nos três ambientes. Só o <code>--env-file</code> muda

Os três recursos que a atividade pede

Recurso	Onde está	Para que serve
Volume	<code>mssql-data:/var/opt/mssql</code>	Os dados do banco sobrevivem ao <code>docker compose down</code> e à recriação do container
Rede	<code>energia-net</code> , uma bridge dedicada	API e banco se acham pelo nome do serviço (<code>Server=mssql,1433</code>) e ficam isolados de outras stacks
Variáveis de ambiente	<code>.env</code> , <code>.env.staging</code> , <code>.env.production</code>	Porta, nome do banco, <code>ASPNETCORE_ENVIRONMENT</code> , senha e chave JWT fora do código

O `depends_on: condition: service_healthy` cuida da ordem de subida. A API só inicia depois que o SQL Server responde na 1433, o que evita aquele erro de migration em container que sobe mais rápido que o banco.

Comandos usados

```
# ambiente local
cp .env.example .env
docker compose up -d --build
bash scripts/smoke-test.sh http://localhost:8080

# staging (API na 8081, banco na 14331)
docker compose --env-file .env.staging -p energia-staging up -d --build
bash scripts/smoke-test.sh http://localhost:8081

# producao (API na 8082, banco na 14332)
docker compose --env-file .env.production -p energia-prod up -d --build
bash scripts/smoke-test.sh http://localhost:8082

# operacao
docker compose logs -f api
docker compose ps
docker compose down -v
```

A imagem

```
$ docker images energiaapi
IMAGE          ID                DISK USAGE  CONTENT SIZE
energiaapi:local  ede2cce21658      375MB        94.9MB
```

No pipeline, a mesma imagem é publicada no GHCR como `ghcr.io/henriquecole/energiaapi:<sha-curto>` e `:latest`.

4. Evidências

Abaixo estão as saídas de terminal da execução local. O arquivo completo está em `docs/evidencias-locais.txt`. Os espaços tracejados são para as capturas de tela do pipeline no GitHub Actions.

4.1 Build e testes

```
$ dotnet test
Test run for EnergiaApi.Tests/bin/Debug/net8.0/EnergiaApi.Tests.dll (.NETCoreApp,Version=v8.0)
A total of 1 test files matched the specified pattern.

Passed! - Failed: 0, Passed: 6, Skipped: 0, Total: 6, Duration: 586 ms
```

Print 1: pipeline completo

Aba Actions com os 4 jobs verdes
`docs/prints/01-pipeline.png`

Print 2: job build-test

Log do build em Release e do `dotnet test` com Passed: 6
`docs/prints/02-build.png` / `docs/prints/03-testes.png`

Print 3: imagem no GHCR

Aba Packages do repositório com as tags publicadas
`docs/prints/04-ghcr.png`

4.2 Os dois ambientes rodando

```
$ docker ps --filter name=energia --format "table {{.Names}}\t{{.Ports}}\t{{.Status}}"
NAMES                                PORTS                                STATUS
energia-prod-api-1                   0.0.0.0:8082->8080/tcp              Up 22 seconds
energia-prod-mssql-1                 0.0.0.0:14332->1433/tcp              Up 27 seconds (healthy)
energia-staging-api-1                0.0.0.0:8081->8080/tcp              Up 55 seconds
energia-staging-mssql-1              0.0.0.0:14331->1433/tcp              Up About a minute (healthy)

$ docker volume ls --filter name=energia
local      energia-prod_mssql-data
local      energia-staging_mssql-data

$ docker network ls --filter name=energia
energia-prod_energia-net    bridge    local
energia-staging_energia-net bridge    local
```

Staging e produção no ar ao mesmo tempo, cada um com seu container, sua rede e seu volume. Isso é resultado do `-p`, que define o nome do projeto, combinado com o `--env-file`.

4.3 Smoke test em staging

```
$ bash scripts/smoke-test.sh http://localhost:8081
==> Aguardando http://localhost:8081/health (timeout 180s)
    API respondeu em 0s
==> 1/4 GET /health
{"status":"ok","ambiente":"Staging","versao":"local"}
==> 2/4 POST /api/auth/login
    token obtido (349 caracteres)
==> 3/4 GET /api/consumos (prova que a API alcanca o banco)
    ok
==> 4/4 GET /api/alertas (prova que o JWT funciona)
    ok
==> SMOKE TEST PASSOU em http://localhost:8081
```

4.4 Smoke test em produção

```
$ bash scripts/smoke-test.sh http://localhost:8082
==> Aguardando http://localhost:8082/health (timeout 180s)
    API respondeu em 0s
==> 1/4 GET /health
{"status":"ok","ambiente":"Production","versao":"local"}
==> 2/4 POST /api/auth/login
    token obtido (349 caracteres)
==> 3/4 GET /api/consumos (prova que a API alcanca o banco)
    ok
==> 4/4 GET /api/alertas (prova que o JWT funciona)
    ok
==> SMOKE TEST PASSOU em http://localhost:8082
```

O campo `ambiente` do `/health` mostra qual instância respondeu: **Staging** na 8081 e **Production** na 8082.

Print 4: deploy em staging

Job deploy-staging com o smoke test passando
docs/prints/05-deploy-staging.png

Print 5: deploy em produção

Job deploy-production, com a aprovação manual registrada
docs/prints/06-deploy-producao.png

Print 6: ambientes no ar

/health e Swagger nas portas 8081 e 8082
docs/prints/07-health-staging.png / docs/prints/08-health-producao.png

5. O que deu trabalho

Problema	O que estava acontecendo	Como resolvi
SQL Server no Mac	A imagem <code>mssql/server</code> não tem build para ARM64 e o container não subia na minha máquina.	Troquei para <code>mcr.microsoft.com/azure-sql-edge</code> , que tem imagem ARM64 e x86. Com isso a mesma stack roda no meu Mac e no runner Linux do GitHub.
A API subia antes do banco	As migrations rodam no startup e falhavam porque o SQL Server ainda estava inicializando.	<code>healthcheck</code> no serviço <code>mssql</code> batendo na porta 1433, mais <code>depends_on: condition: service_healthy</code> na API. O <code>EnableRetryOnFailure()</code> do EF Core cobre o resto.
Migrations quebrando os testes	Os testes usam banco InMemory, que não aceita <code>Database.Migrate()</code> .	Um <code>if (!app.Environment.IsEnvironment("Testing"))</code> no <code>Program.cs</code> . A <code>ApiFactory</code> dos testes sobe com <code>UseEnvironment("Testing")</code> .
Senha dentro do compose	A senha do banco e a chave JWT estavam fixas no <code>docker-compose.yml</code> e no <code>appsettings.json</code> .	Tudo virou variável de ambiente, com <code>\${DB_PASSWORD:?..}</code> falhando na hora se faltar. Em CI os valores vêm dos secrets. O <code>.env</code> entrou no <code>.gitignore</code> e só o <code>.env.example</code> ficou versionado.
Dois ambientes na mesma máquina	Staging e produção brigavam por porta, nome de container e volume.	Um compose só, parametrizado, com um <code>--env-file</code> por ambiente (8081 e 14331 contra 8082 e 14332) e o <code>-p</code> separando containers, rede e volume.
Nome da imagem no GHCR	O push falhava com erro de nome inválido, porque <code>github.repository</code> vem com letra maiúscula e o GHCR só aceita minúscula.	Um passo que normaliza o nome com <code>REPO="\${GITHUB_REPOSITORY,,}"</code> e expõe a tag como output do job.
Healthcheck dentro do container	A imagem <code>aspnet:8.0</code> não tem <code>curl</code> nem <code>wget</code> , então um healthcheck HTTP no compose exigiria instalar pacote e engordar a imagem.	Deixei a imagem enxuta e passei a validar de fora, com o <code>scripts/smoke-test.sh</code> . É o mesmo script que roda no pipeline e na minha máquina.
Provar qual ambiente respondeu	O Swagger só era registrado em <code>Development</code> , e nada na resposta dizia em qual ambiente a API estava	Criei o <code>/health</code> devolvendo <code>ambiente</code> e <code>versao</code> , que é o SHA do commit, e liberei o Swagger também em <code>Staging</code> .

	rodando. Ficava difícil evidenciar isso num print.	
Bit de execução no ZIP	Ao empacotar o projeto, o <code>smoke-test.sh</code> perdia a permissão de execução. Quem extraísse o ZIP e desse push teria o pipeline falhando com "permission denied".	O workflow chama <code>bash scripts/smoke-test.sh</code> em vez de <code>./scripts/smoke-test.sh</code> , e o empacotamento preserva o modo do arquivo.

6. Checklist e stack

Item	OK
Projeto compactado em .ZIP com estrutura organizada	Sim
Dockerfile funcional	Sim
docker-compose.yml ou arquivos Kubernetes	Sim
Pipeline com etapas de build, teste e deploy	Sim
README.md com instruções e prints	Sim
Documentação técnica com evidências (PDF ou PPT)	Sim
Deploy realizado nos ambientes staging e produção	Sim

Tecnologias usadas

Camada	Stack
Aplicação	ASP.NET Core 8 em MVVM, Entity Framework Core 8 com migrations, JWT Bearer, Swagger e OpenAPI
Banco de dados	SQL Server, na imagem Azure SQL Edge, com volume persistente
Frontend	React, Vite e TypeScript
Testes	xUnit, <code>Microsoft.AspNetCore.Mvc.Testing</code> , EF Core InMemory, Bash e curl no smoke test, Postman
Container	Docker com build em dois estágios, Docker Compose com volume, rede e variáveis
CI/CD	GitHub Actions, GitHub Environments para segredos e aprovação, GitHub Container Registry, cache de build <code>type=gha</code>